SSE 2105
ADVANCED OBJECT-ORIENTED PROGRAMMING

Lecturer's Notes

Bachelor of Science in Software Engineering

Course Orientation

These notes are summarized and they follow the SSE 2105 course outline. The sequence is: object-oriented programming paradigm; introduction to Java; control structures; functions and arrays; objects and classes; inheritance; polymorphism; encapsulation and abstraction; Java GUI input/output; Java applets; and advanced Java programming through database connectivity.

Course Learning Outcomes

1. Explain the syntax and semantics of the Java programming language.
2. Explain and apply the concepts of classes and objects.
3. Design, code and implement simple to intermediate standalone object-oriented applications.

Teaching Approach

- Blended learning using classroom and online activities.
- Case analysis and worked programming examples.
- Group discussion and Topicly topic summaries.
- Laboratory practice and programming exercises.

Course Schedule

Topic	Topic	Main Coverage
1	Introduction to OOP Paradigm	Encapsulation, abstraction, classes, objects, polymorphism, inheritance, data hiding, OOP languages, merits/demerits, procedural vs OOP
2	Introduction to Java Programming	Java overview, setup, identifiers, keywords, data types, operators, expressions, namespace/package idea, casting, I/O
3	Java Control Structures	if, if-else, nested if, for, while, do-while, nested loops
4	Java Functions and Arrays	Methods, parameters, calls, 1D and multidimensional arrays
5	Objects and Classes	Classes, data members, member functions, access specifiers, object creation
6	CAT One / Assignment One	Assessment of Topics 1-5
7	Java Inheritance	Introduction, inheritance types, implementation
8	Java Polymorphism	Overloading, overriding, implementation of polymorphism
9	Encapsulation and Abstraction	Encapsulation, access control, abstraction approaches
10	Java Input/Output and GUI	Creating GUIs, GUI components, containers, event handling
11	CAT Two / Assignment Two	Assessment
12	Java Applets	Legacy applet concepts, strings, lines and basic shapes, web embedding concept
13	Advanced Java Programming	JDBC connection, insert, retrieve, update and delete
14	Revision	Revision for examinations
15	End-Semester Examination	Final assessment

TOPIC 1: INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING PARADIGM

Learning Outcomes

By the end of this topic, you should be able to:

- Define object-oriented programming and its key terminology.
- Differentiate classes from objects.
- Explain encapsulation, abstraction, inheritance, polymorphism and data hiding.
- Compare procedural programming with object-oriented programming.
- State merits and limitations of OOP.

1.1 Programming Paradigms

A programming paradigm is a general style of organizing the logic and structure of software. In procedural programming, the program is primarily decomposed into procedures or functions. In object-oriented programming, the program is modeled as interacting objects that combine state and behaviour.

1.2 Class and Object

A class is a blueprint that defines the attributes and behaviours that its objects possess. An object is an instance of a class.

```
class Student {
    String registrationNumber;
    String name;

    void displayDetails() {
        System.out.println(registrationNumber + " - " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.registrationNumber = "SSE001";
        s1.name = "Amina";
        s1.displayDetails();
    }
}
```

1.3 Encapsulation and Data Hiding

Encapsulation bundles data and the methods that operate on that data inside a class. Data hiding restricts direct access to internal state, commonly by declaring fields private and exposing controlled methods.

```
class BankAccount {
    private double balance;

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
```

```
    }

    public double getBalance() {
        return balance;
    }
}
```

1.4 Abstraction

Abstraction exposes essential operations while hiding unnecessary implementation details.

```
abstract class Shape {
    abstract double area();
}

class Circle extends Shape {
    private double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    @Override
    double area() {
        return Math.PI * radius * radius;
    }
}
```

1.5 Inheritance

Inheritance allows a new class to reuse and specialize accessible features of an existing class.

```
class Person {
    String name;
}

class Student extends Person {
    String registrationNumber;
}
```

1.6 Polymorphism

Polymorphism allows the same operation to produce behaviour appropriate to the actual object.

```
class Animal {
    void makeSound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
```

```
    @Override
    void makeSound() {
        System.out.println("Cat meows");
    }
}

public class TestAnimals {
    public static void main(String[] args) {
        Animal a1 = new Dog();
        Animal a2 = new Cat();
        a1.makeSound();
        a2.makeSound();
    }
}
```

1.7 OOP Languages

- Java
- C++
- C#
- Python (supports OOP among other paradigms)
- Ruby

1.8 Merits and Demerits of OOP

Merits	Possible limitations
Promotes modularity through classes.	Can introduce more design overhead for very small programs.
Supports reuse through inheritance and composition.	Poorly designed inheritance can increase coupling.
Encapsulation protects object state.	Requires understanding of object relationships.
Polymorphism supports extensibility.	Runtime dispatch and object creation may add overhead in some contexts.

1.9 Procedural versus OOP

Aspect	Procedural	Object-Oriented
Primary unit	Procedure/function	Class/object
Organization	Operations around data	Data and behaviour together
Reuse	Functions/modules	Classes, composition, inheritance
Typical design focus	Sequence of operations	Interacting objects

Common Programming Errors

- Confusing a class with an object. A class defines a type; an object is an instance.
- Assuming encapsulation means only using getters and setters. Its purpose is controlled access and protection of invariants.
- Using inheritance only for code reuse without an appropriate 'is-a' relationship.

Laboratory / Practical Exercise

Create a small object model for a university system.

1. Create a Student class with registration number, name and programme.
2. Create two Student objects and display their details.
3. Make at least one field private and provide controlled access.

4. Create a Person parent class and let Student inherit from it.

Review Questions

1. Differentiate a class from an object.
2. Explain encapsulation and data hiding with an example.
3. Explain inheritance and polymorphism.
4. Give four merits of OOP.
5. Compare procedural programming and OOP.

TOPIC 2: INTRODUCTION TO JAVA PROGRAMMING

Learning Outcomes

By the end of this topic, you should be able to:

- Explain the role of Java, the compiler and the JVM.
- Write, compile and execute a simple Java application.
- Use identifiers, variables, primitive data types and operators.
- Apply type casting and basic keyboard/output statements.

2.1 Java Program Execution

A Java source file is compiled into bytecode. The Java Virtual Machine executes the bytecode on the target platform.

```
Source (.java) -> javac compiler -> Bytecode (.class) -> JVM -> Operating system
```

2.2 First Java Program

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

The file name for a public class should match the public class name, for example HelloWorld.java.

2.3 Identifiers and Keywords

Identifiers name program elements such as classes, variables and methods. They cannot begin with a digit, cannot contain spaces and cannot be Java keywords. Java is case-sensitive.

```
int studentAge = 21;     // valid
String studentName = "Ali";
// int 2age = 21;         // invalid
// int class = 5;         // invalid: class is a keyword
```

2.4 Primitive Data Types

Type	Purpose	Example
byte, short, int, long	Whole numbers	int age = 20;
float, double	Real numbers	double average = 67.5;
char	Single UTF-16 code unit	char grade = 'A';
boolean	true/false	boolean active = true;

2.5 Operators and Expressions

```
int x = 10;
int y = 3;

System.out.println(x + y); // 13
System.out.println(x - y); // 7
System.out.println(x * y); // 30
System.out.println(x / y); // 3 (integer division)
System.out.println(x % y); // 1
```

```
boolean result = x > y && y != 0;
System.out.println(result);
```

2.6 Type Casting

```
int count = 25;
double widened = count;       // widening conversion

double price = 99.95;
int whole = (int) price;      // explicit narrowing
System.out.println(whole);    // 99
```

2.7 Basic Input and Output

```
import java.util.Scanner;

public class StudentInput {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter name: ");
        String name = input.nextLine();

        System.out.print("Enter age: ");
        int age = input.nextInt();

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);

        input.close();
    }
}
```

2.8 Packages and Namespaces

Java uses packages to organize related classes and help avoid naming conflicts. The supplied outline uses the term namespace; in Java teaching, packages are the closest direct language mechanism for organizing named types.

```
package university.students;

import java.util.Scanner;
```

Common Programming Errors

- Using double quotes for char values instead of single quotes.
- Comparing strings with == instead of equals() when value comparison is intended.
- Forgetting that integer division discards the fractional part.
- Using a variable before it has been initialized.

Laboratory / Practical Exercise

Develop a Java console application that captures and displays student information.

1. Read registration number, name, age and average mark.

2. Store each value in an appropriate data type.
3. Calculate the student's age next year.
4. Display all values in a formatted output.

Review Questions

1. Differentiate JDK, compiler bytecode and JVM.
2. State the rules for Java identifiers.
3. Differentiate widening and narrowing conversions.
4. Explain the result of 10 / 3 when both operands are int.
5. Write a program that reads two numbers and displays their sum, product and average.

TOPIC 3: JAVA CONTROL STRUCTURES

Learning Outcomes

By the end of this topic, you should be able to:

- Use selection statements to control program flow.
- Apply for, while and do-while loops.
- Construct nested decisions and nested loops.
- Choose an appropriate control structure for a given problem.

3.1 Selection using if

```
int marks = 68;

if (marks >= 50) {
    System.out.println("Pass");
}
```

3.2 if...else and else-if Ladder

```
int marks = 74;

if (marks >= 70) {
    System.out.println("Grade A");
} else if (marks >= 60) {
    System.out.println("Grade B");
} else if (marks >= 50) {
    System.out.println("Grade C");
} else if (marks >= 40) {
    System.out.println("Grade D");
} else {
    System.out.println("Fail");
}
```

3.3 Nested if

```
int age = 22;
boolean registered = true;

if (age >= 18) {
    if (registered) {
        System.out.println("Eligible to access the portal.");
    } else {
        System.out.println("Complete registration first.");
    }
}
```

3.4 for Loop

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Iteration " + i);
}
```

3.5 while Loop

```
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
```

3.6 do...while Loop

```
int i = 1;
do {
    System.out.println(i);
    i++;
} while (i <= 5);
```

3.7 Nested Loops

```
for (int row = 1; row <= 4; row++) {
    for (int col = 1; col <= 5; col++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

3.8 Worked Example: Average of Repeated Marks

```
import java.util.Scanner;

public class AverageMarks {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        double total = 0;

        for (int i = 1; i <= 5; i++) {
            System.out.print("Enter mark " + i + ": ");
            total += input.nextDouble();
        }

        double average = total / 5;
        System.out.println("Average = " + average);
        input.close();
    }
}
```

Common Programming Errors

- Placing a semicolon immediately after an if or loop condition unintentionally.
- Creating an infinite loop by failing to update the loop-control variable.
- Using = instead of == in contexts where comparison is intended.
- Off-by-one errors such as using <= array.length when indexing an array.

Laboratory / Practical Exercise

Create a menu-driven grade program.

1. Read a mark and display its grade using if/else-if.

2. Use a loop to process five students.
3. Count how many students pass and fail.
4. Display the class average.

Review Questions

1. Differentiate while and do-while.
2. Write an if statement that tests whether a number is positive, negative or zero.
3. Write a for loop that prints even numbers from 2 to 20.
4. Explain a nested loop.
5. Identify two causes of infinite loops.

TOPIC 4: JAVA FUNCTIONS (METHODS) AND ARRAYS

Learning Outcomes

By the end of this topic, you should be able to:

- Declare, define and call Java methods.
- Pass arguments to methods and return values.
- Create and process one-dimensional arrays.
- Create and process multidimensional arrays.

4.1 Methods

In Java, executable functions belong to classes and are normally called methods.

```
static int add(int a, int b) {
    return a + b;
}

public static void main(String[] args) {
    int answer = add(10, 20);
    System.out.println(answer);
}
```

4.2 Parameters and Arguments

```
static void displayStudent(String name, int age) {
    System.out.println(name + " is " + age + " years old.");
}

displayStudent("Amina", 21);
```

Java passes arguments by value. For primitive values, the method receives a copy of the value. For object references, the reference value is copied, so the method can use that copied reference to modify the referenced object's mutable state.

4.3 One-Dimensional Arrays

```
int[] marks = {67, 78, 56, 89, 73};

int total = 0;
for (int mark : marks) {
    total += mark;
}

double average = (double) total / marks.length;
System.out.println("Average = " + average);
```

4.4 Array Input

```
import java.util.Scanner;

public class ArrayInput {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int[] marks = new int[5];
```

```
        for (int i = 0; i < marks.length; i++) {
            System.out.print("Enter mark: ");
            marks[i] = input.nextInt();
        }

        for (int mark : marks) {
            System.out.println(mark);
        }

        input.close();
    }
}
```

4.5 Multidimensional Arrays

```
int[][] marks = {
    {70, 80, 90},
    {60, 75, 85},
    {55, 65, 75}
};

for (int row = 0; row < marks.length; row++) {
    for (int col = 0; col < marks[row].length; col++) {
        System.out.print(marks[row][col] + " ");
    }
    System.out.println();
}
```

Common Programming Errors

- Using an array index outside the range 0 to length - 1.
- Forgetting to cast when a floating-point average is required from integer totals.
- Declaring a method inside another method. Java methods are declared in classes, not nested inside methods.
- Assuming Java object arguments are passed by reference. Java is pass-by-value.

Laboratory / Practical Exercise

Build a marks analysis application.

1. Store ten marks in an array.
2. Create methods to calculate total, average, maximum and minimum.
3. Display all marks using an enhanced for loop.
4. Extend the program to a 2D array representing students and units.

Review Questions

1. What is a method?
2. Differentiate a parameter from an argument.
3. Explain Java's pass-by-value semantics.
4. Write a method that returns the larger of two integers.
5. Write code to find the maximum value in an integer array.

TOPIC 5: ASSIGNMENT ONE

Suggested assessment coverage: OOP concepts, Java fundamentals, control structures, methods and arrays.

Practical Assignment

Student Grade Management Console Application

1. Capture a student's registration number and name.
2. Read marks for at least five units into an array.
3. Use methods to calculate total and average.
4. Use selection to assign a grade.
5. Use loops to display the marks.
6. Validate that each mark is between 0 and 100.

TOPIC 6: OBJECTS AND CLASSES

Learning Outcomes

By the end of this topic, you should be able to:

- Define a class and create objects.
- Use fields, methods and access modifiers.
- Initialize objects with constructors.
- Use the this keyword appropriately.

6.1 Defining a Class

```
class Student {
    String registrationNumber;
    String name;
    String programme;

    void display() {
        System.out.println(registrationNumber);
        System.out.println(name);
        System.out.println(programme);
    }
}
```

6.2 Creating Objects

```
Student s1 = new Student();
s1.registrationNumber = "SSE001";
s1.name = "Amina";
s1.programme = "Software Engineering";
s1.display();
```

6.3 Constructors

```
class Student {
    private String registrationNumber;
    private String name;

    Student(String registrationNumber, String name) {
        this.registrationNumber = registrationNumber;
        this.name = name;
    }

    void display() {
        System.out.println(registrationNumber + " " + name);
    }
}
```

6.4 Access Specifiers

Modifier	Typical access	Use
private	Within the declaring class	Hide implementation/state
default	Within the same package	Package-level collaboration
protected	Same package and subclasses	Inheritance-related access

	subject to Java access rules	
public	Where the member's declaring type is accessible	Public API

6.5 Complete Example

```
class Course {
    private String code;
    private String title;

    Course(String code, String title) {
        this.code = code;
        this.title = title;
    }

    public String getCode() {
        return code;
    }

    public void display() {
        System.out.println(code + ": " + title);
    }
}

public class CourseTest {
    public static void main(String[] args) {
        Course c = new Course("SSE2105", "Advanced OOP");
        c.display();
    }
}
```

Common Programming Errors

- Forgetting to use new when creating an object.
- Writing a return type for a constructor.
- Confusing local variables with instance variables.
- Making all fields public and thereby weakening encapsulation.

Laboratory / Practical Exercise

Model a library using classes.

1. Create a Book class with ISBN, title and author.
2. Create a constructor that initializes all fields.
3. Create a displayDetails() method.
4. Create at least three Book objects in a test class.

Review Questions

1. Define class, object, field and method.
2. What is a constructor?
3. Explain the purpose of this.
4. Differentiate private and public members.
5. Write a class Employee with name and salary fields and a method to display them.